[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

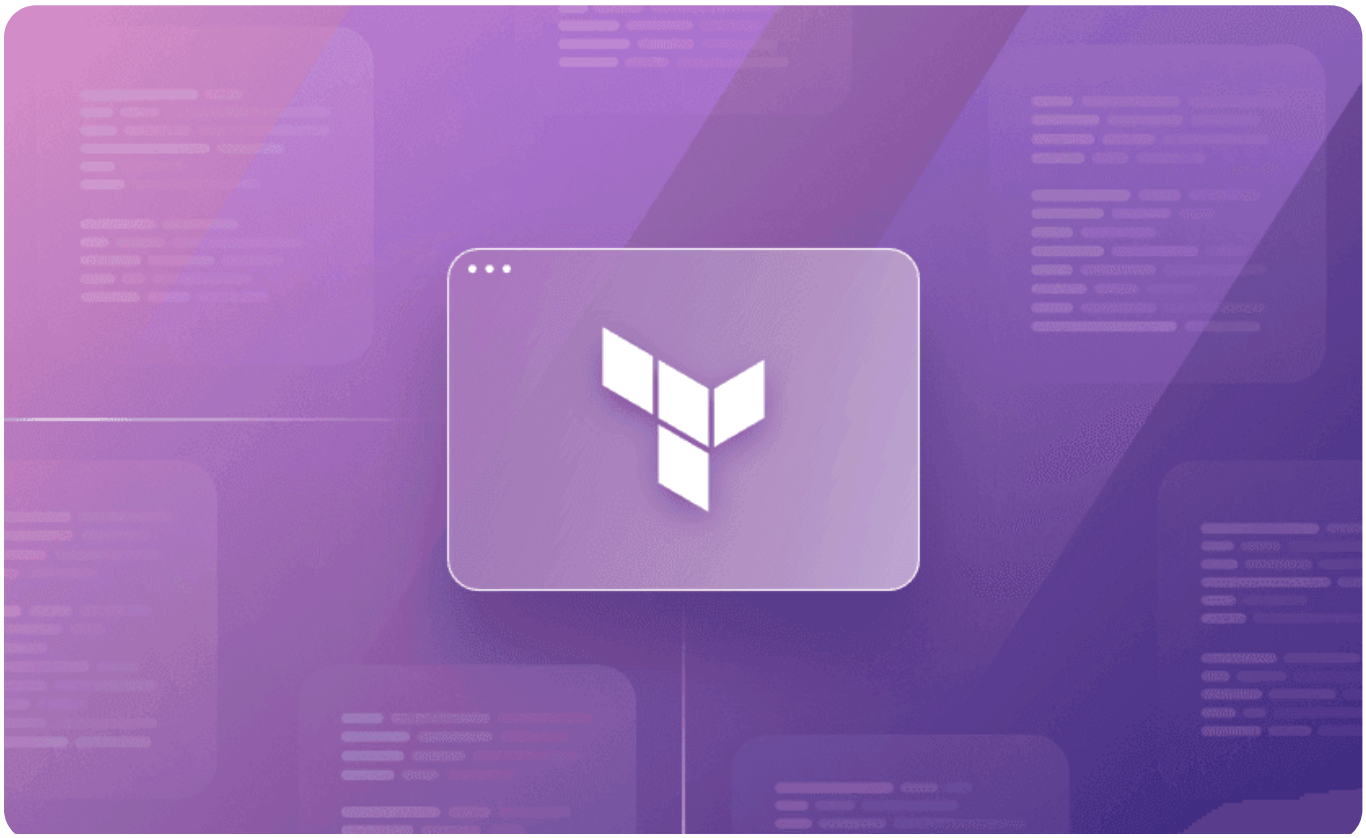
TERRAFORM

Terraform Architecture Overview – Structure and Workflow



Vivekanand Rapaka

Updated 19 Dec 2025 · 12 min read



Terraform is an infrastructure-as-code (IaC) tool developed by Hashicorp that enables you to provision your infrastructure in a simple, efficient, and declarative manner through repeatable code.

One of Terraform's key features is its cloud-agnostic nature; you can deploy infrastructure to any cloud environment, such as Azure, AWS, Google Cloud, and VMware, as well as on-premises environments. This benefit allows you to automate your cloud infrastructure without having to learn or adopt different tools for each cloud service provider.

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

1. What is the architecture of Terraform?
2. Basic components of Terraform architecture
3. How to structure Terraform files
4. Terraform workflow
5. How does Terraform architecture compare to other IaC tools

What is the architecture of Terraform?

Terraform has a **modular and client-server architecture** designed to manage infrastructure as code efficiently. At its core, Terraform uses configuration files written in HashiCorp Configuration Language (HCL) to define desired infrastructure resources.

The Terraform CLI acts as the client, executing commands like plan, apply, and destroy to manage the lifecycle of infrastructure.

A key component is the Terraform state file, which records the current state of managed resources. This enables Terraform to track and compare changes over time. Terraform also supports remote backends to store state files securely and enable team collaboration.

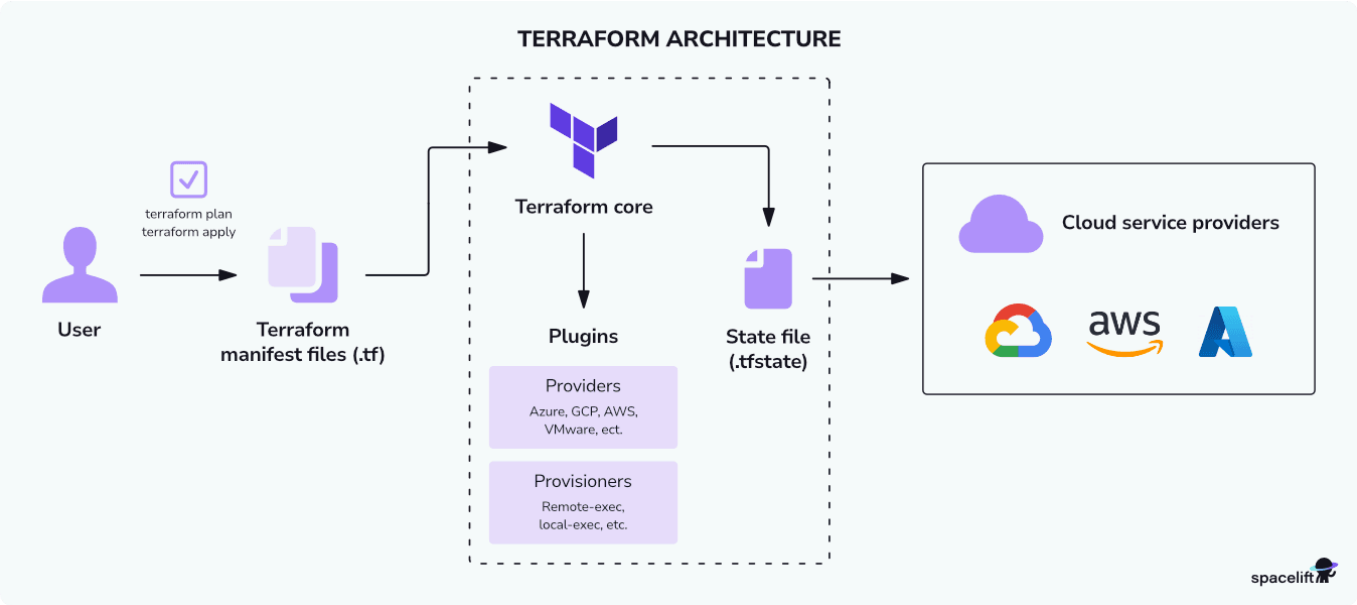
Note: New versions of Terraform are placed under the BUSL license, but everything created before version 1.5.x stays open-source. OpenTofu is an open-source version of Terraform that expands on Terraform's existing concepts and offerings. It is a viable alternative to HashiCorp's Terraform, being forked from Terraform version 1.5.6.

Basic components of Terraform architecture

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

- **Terraform core** is the engine that reads configuration files, manages state, and executes plans to apply changes.
- **Providers** are responsible for interacting with APIs of services like AWS, Azure, or Kubernetes to provision resources.
- **Configuration files**, written in HashiCorp Configuration Language (HCL), define the desired infrastructure in a human-readable format.
- **State management** tracks the current infrastructure setup, allowing Terraform to compare it with the desired state and determine necessary changes.

Below you can see the Terraform architecture diagram:



1. Terraform core

Terraform's core (also known as Terraform CLI) is built on a statically compiled binary developed using the Go programming language.

This binary generates the command-line tool (CLI) "terraform," which serves as the primary interface for Terraform users. It can be accessed on the Terraform GitHub

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

Terraform providers are modules that enable Terraform to communicate with a diverse range of services and resources, including but not limited to cloud providers, databases, and DNS services.

Each provider is responsible for defining the resources that Terraform can manage within a particular service and translating Terraform configurations into API calls that are specific to that service.

Providers are available for numerous services and resources, including those developed by major cloud providers like AWS, Azure, and Google Cloud, as well as community-supported providers for various services. By utilizing providers, Terraform users can maintain their infrastructure consistently and reproducibly, regardless of the underlying service or provider.

3. State file

The Terraform state file is an essential aspect of Terraform's functionality. It is a JSON file that stores information about the resources that Terraform manages, including their current state and dependencies.

Terraform utilizes the state file to determine the changes that need to be made to the infrastructure when a new configuration is applied. It also ensures that resources are not unnecessarily recreated across multiple runs of Terraform.

The state file can be kept locally on the machine running Terraform or remotely using a remote backend like Azure Storage Account or Amazon S3, or HashiCorp Consul. It is crucial to safeguard the state file and maintain frequent backups since it contains sensitive information about the infrastructure being managed.

4. Configuration files

Terraform's structure is based on modular, human-readable configuration files written in HashiCorp Configuration Language (HCL). These files define infrastructure components

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

Azure) Terraform should interact with. Resources represent the actual infrastructure to be created, such as virtual machines or databases. Variables enable reusability and flexibility, while outputs expose key information after deployment, such as IP addresses.

How to structure Terraform files

Declaring resources is very easy in Terraform. Terraform files always end with the extension `.tf`.

The basic Terraform file structure contains the following elements.

Terraform block

A Terraform block specifies the required providers for Terraform to execute the script. This block also contains the source block, which specifies where Terraform should download the provider and the required version.

Below is an example:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "=3.0.0"
    }
  }
}
```

Provider block

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

For example, if you are using Azure as your service provider, it would look as follows:

```
provider "azurerm" {
  features {}
  subscription_id = "00000000-0000-0000-0000-000000000000"
  tenant_id = "11111111-1111-1111-1111-111111111111"
}
```

Resource block

A resource block represents a particular resource in the cloud provider's services. It includes the resource type, name, and configuration details. This is the main block that specifies the type of resource we are trying to deploy.

Below is an example of creating a resource group in Azure:

```
resource "azurerm_resource_group" "example" {
  name = "example"
  location = "West Europe"
}
```

Data block

A data block fetches data from the provider's services, which can be used in resource blocks. It includes the data type and configuration details.

This is used in scenarios where the resource has already been deployed and you would like to retrieve its details.

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

```
data "azurerm_resource_group" "example" {
  name = "existing"
}
```

Variable block

A variable block defines input variables for the Terraform configuration. It includes the variable name, type, and default value.

The following is an example of a variable block in Terraform:

```
variable "resource_group_name" {
  default = "myTFResourceGroup"
}
```

Output block

An output block defines the output values generated by the Terraform configuration. It includes the output name and value.

```
output "resource_group_id" {
  value = azurerm_resource_group.rg.id
}
```

Provisioners

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

software, running tests, and performing any other necessary actions.

Provisioners are executed after a resource has been created and can also be triggered when a resource is destroyed. Terraform comes with several built-in provisioners, including the file provisioner for copying files to a resource and the remote-exec provisioner for running commands on a remote machine. It is also possible to create custom provisioners for more advanced use cases.

This is mainly used as a last resort unless you are unable to achieve your desired outcome with existing resource blocks in Terraform.



Terraform workflow

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

The following are the general steps involved in the lifecycle of resource creation:

1. **Write:** Author your Infrastructure as Code in a Terraform configuration file with all the required blocks.
2. **Init**: Initialize the Terraform working directory and download any necessary plugins. This is usually done using `terraform init` command.
3. **Plan:** Review the Terraform execution plan to see what changes will be made to the infrastructure. This is done using the `terraform plan` command. Running this command will show the actions that Terraform is going to perform when `terraform apply` command is run.
4. **Apply:** Apply the changes to create or modify the infrastructure. Once you are comfortable with the changes shown in the output of `terraform plan` command, you can apply those changes using `terraform apply` command.

If you need to apply a new configuration, make the necessary changes in the configuration and follow the steps above to apply it.

These steps are often repeated as part of a continuous integration/continuous delivery (CI/CD) pipeline or as part of ongoing infrastructure management. Terraform's declarative approach to infrastructure as code allows for a consistent and reproducible workflow, where infrastructure changes can be tracked, reviewed, and audited over time.

If you want to have all the Terraform commands in one place, check out our Terraform Cheat Sheet.

💡 You might also like:

- 5 Ways to Manage Terraform at Scale
- How to Automate Terraform Deployments and Infrastructure Provisioning

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

How does Terraform architecture compare to other IaC tools

Terraform's architecture is built around a declarative model, meaning users define the desired end state of their infrastructure, and Terraform determines the necessary steps to achieve it.

Central to its design is the use of a state file, which tracks existing resources and enables Terraform to detect changes and plan updates accurately before applying them. This "plan and apply" approach helps prevent unintended modifications and supports automated workflows.

Unlike **Ansible** or **Chef**, which follow an imperative or procedural approach — executing step-by-step instructions — Terraform focuses on describing *what* infrastructure should look like, not *how* to build it. (See: Ansible vs. Terraform)

Additionally, whereas tools like Ansible often require agents or SSH access to remote systems, Terraform is agentless and interacts directly with cloud APIs, which improves security and simplifies deployment.

Compared to **cloud-native IaC tools** like AWS CloudFormation or Azure Resource Manager (ARM) templates, Terraform is cloud-agnostic. It supports a wide range of providers, including AWS, Azure, Google Cloud, Kubernetes, and even on-prem solutions. This gives teams greater flexibility and avoids vendor lock-in. (See: Terraform vs. AWS CloudFormation)

Deploying Terraform resources with Spacelift

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

unlocking features such as:

- **Policies (based on Open Policy Agent)** – You can control how many approvals you need for runs, what kind of resources you can create, and what kind of parameters these resources can have, and you can also control the behavior when a pull request is open or merged.
- **Multi-IaC workflows** – Combine Terraform with Kubernetes, Ansible, and other infrastructure-as-code (IaC) tools such as OpenTofu, Pulumi, and CloudFormation, create dependencies among them, and share outputs
- **Build self-service infrastructure** – You can use Blueprints to build self-service infrastructure; simply complete a form to provision infrastructure based on Terraform and other supported tools.
- **Integrations with any third-party tools** – You can integrate with your favorite third-party tools and even build policies for them. For example, see how to integrate security tools in your workflows using Custom Inputs.

Spacelift enables you to create private workers inside your infrastructure, which helps you execute Spacelift-related workflows on your end. Read the documentation for more information on configuring private workers.

You can try it for free by creating a trial account or booking a demo with one of our engineers.

Key points

This blog post covered several key topics related to Terraform and delved into the Terraform template structure and architecture, including the Terraform core and provisioners. Additionally, we discussed the various steps involved in Terraform's workflow.

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

Automate your infrastructure provisioning, and build more complex workflows based on Terraform using policy as code, programmatic configuration, context sharing, drift detection, resource visualization, and many more.

Learn more

Written by



Vivekanand Rapaka

Vivekanand is a seasoned IT professional with expertise in Azure Cloud, Azure DevOps, Terraform, Gitlab, and Kubernetes. He holds multiple certifications from Microsoft, Hashicorp, and CNCF, including the Certified Kubernetes Administrator (CKA) certification. With a passion for exploring new technology trends, he is always on the lookout for the latest developments in the world of DevOps. Vivekanand is committed to staying up-to-date with the latest developments in the industry and is dedicated to sharing his knowledge with the DevOps community through informative blog posts.

blog

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️



TERRAFORM

8 min read

How to Manage On-premise Infrastructure with Terraform

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️



TERRAFORM

7 min read

How to Use the If / Else Statement in Terraform – Examples



TERRAFORM

13 min read

Terraform var file: Variables Management with Examples

[Virtual Event] The AI Readiness gap: Where infrastructure teams are getting AI wrong Register ➡️

Product

How it Works

IaC/GitOps

Platform Capabilities

Intent

Pricing

Platform Security

Documentation

Support Knowledge Base

System Status

Product Changelog

Test Pilot Program

Company

About Us

Careers

Contact Us

Partners

Media resources

Accessibility

Learn

Spacelift Blog

Spacelift vs Atlantis

Spacelift vs Terraform Cloud

Spacelift vs Terraform Enterprise

Customer Stories

Webinars

Upcoming Events

Mission Infrastructure Newsletter

spacelift

Get our newsletter

Subscribe

Privacy Policy Terms of Service Trust Center

© 2026 Spacelift, Inc. All rights reserved